

Ketentuan Tugas Pendahuluan

- Untuk soal teori **JAWABAN DIKETIK DENGAN RAPI** dan untuk soal algoritma **SERTAKAN SCREENSHOOT CODINGAN DAN HASIL OUTPUT**.
- Deadline pengumpulan TP Modul 5 adalah **Senin, 14 Oktober 2024** pukul **06.00 WIB**.
- **TIDAK ADA TOLERANSI KETERLAMBATAN, TERLAMBAT ATAU TIDAK MENGUMPULKAN TP ONLINE MAKA DIANGGAP TIDAK MENGERJAKAN.**
- **DILARANG PLAGIAT (PLAGIAT = E).**
- Kerjakan TP dengan jelas agar dapat dimengerti.
- Untuk setiap soal nama fungsi atau prosedur **WAJIB** menyertakan **NIM**, contoh: **insertFirst_130122xxxx**.
- File diupload di LMS menggunakan format PDF dengan ketentuan : **TP_MODX_NIM_KELAS.pdf** -

Contoh:

```
int searchNode_130122xxxx (List L, int X);
```

CP:

- Raihan (089638482851)
- Kayyisa (085105303555)
- Abiya (082127180662)
- Rio (081210978384)

SELAMAT MENGERJAKAN^^

LAPORAN PRAKTIKUM
MODUL 5
“ Single Linked List”



Disusun Oleh:
Shilfi Habibah - 2311104002
S1SE07-01

Dosen :
Yudha Islami Sulistya

PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
INSTITUT TEKNOLOGI TELKOM PURWOKERTO
2024

SOAL TP

Soal 1: Mencari Elemen Tertentu dalam SLL

Deskripsi Soal: Buatlah program yang mengizinkan pengguna memasukkan 6 elemen integer ke dalam list. Implementasikan function **searchElement** untuk mencari apakah sebuah nilai tertentu ada dalam list.

Instruksi

1. Minta pengguna untuk memasukan nilai yang ingin dicari.
2. Jika nilai ditemukan, tampilkan alamat dan posisi dalam angka (contoh: urutan ke 4) pada list tersebut.
3. Jika nilai tidak ditemukan, tampilkan pesan bahwa elemen tersebut tidak ada dalam list tersebut.

NB:

1. Gunakan pendekatan linier search untuk mencari elemen.

Sub-Program:

```
Function searchElement( L : list, i : integer)
{ I.S. List tidak kosong.
  F.S. Menampilkan alamat dan posisi elemen i jika ditemukan}
Dictionary
    current: address
    position: int
Algorithms
    current ← L.head
    position ← 1

    //melakukan perulangan selama i belum ditemukan dan posisi current belum berada pada
    akhir list
    While .....
        //seiring pointer (current) bergerak, position bertambah
        . . . . .
        //lakukan perpindahan current
        . . . . .
    endwhile
    //jika i ditemukan maka tampilkan alamat dan posisi
    if....
        output(...)
    //jika tidak ditemukan maka tampilkan pesan yang menyatakan hal tsb
    else...
        output(...)
    endif
endfunction
```

Code:

```

#include <iostream>
using namespace std;

// Fungsi untuk mencari elemen menggunakan linear search
void searchElement_2311104002(int L[], int size, int i) {
    int position = 1; // Posisi awal elemen
    bool found = false; // Flag untuk mengecek apakah elemen ditemukan atau tidak

    // Linear search untuk mencari elemen dalam array
    for (int index = 0; index < size; index++) {
        if (L[index] == i) {
            found = true;
            cout << "Elemen ditemukan pada posisi urutan ke-" << position
                  << ", alamat: " << &L[index] << endl;
            break;
        }
        position++;
    }

    // Jika elemen tidak ditemukan
    if (!found) {
        cout << "Elemen " << i << " tidak ditemukan dalam list." << endl;
    }
}

int main() {
    int L[6]; // Array untuk menampung 6 elemen integer

    // Memasukkan 6 elemen integer ke dalam array
    for (int i = 0; i < 6; i++) {
        cout << "Masukkan elemen integer: ";
        cin >> L[i];
    }

    // Meminta pengguna untuk memasukkan nilai yang ingin dicari
    int searchVal;
    cout << "Masukkan nilai yang ingin dicari: ";
    cin >> searchVal;

    // Panggil fungsi searchElement
    searchElement_2311104002(L, 6, searchVal);

    return 0;
}

```

- Fungsi **searchElement(int L[], int size, int i)** : Mencari elemen i dalam array L dengan pendekatan linear search, Jika ditemukan akan muncul posisi elemen dalam urutan serta alamat memori (dengan &L[index] berfungsi mengambil alamat memori)

Output:

```

Masukkan elemen integer: 10
Masukkan elemen integer: 15
Masukkan elemen integer: 20
Masukkan elemen integer: 25
Masukkan elemen integer: 30
Masukkan elemen integer: 35
Masukkan nilai yang ingin dicari: 25
Elemen ditemukan pada posisi urutan ke-4, alamat: 0x61fe0c

```

- Program melakukan pencarian linier sesuai pengguna memasukkan 6 elemen integer array L, Setelah itu masukkan nilai yang ingin dicari dan menampilkan pesan bahwa elemen tersebut tidak ada dalam list atau tidak, Jika ada akan muncul juga urutannya.

Soal 2: Mengurutkan List Menggunakan Bubble Sort

Deskripsi Soal: Buatlah program yang mengizinkan pengguna memasukkan 5 elemen integer ke dalam list. Implementasikan procedure **bubbleSortList** untuk mengurutkan elemen-elemen dalam list dari nilai terkecil ke terbesar.

Instruksi

Setelah mengurutkan, tampilkan elemen-elemen list dalam urutan yang benar.

Langkah-langkah Bubble Sort pada SLL

1. Inisialisasi:
 - Buat pointer current yang akan digunakan untuk menelusuri list.
 - Gunakan variabel boolean swapped untuk mengawasi apakah ada pertukaran yang dilakukan pada iterasi saat ini.
2. Traversing dan Pertukaran:
 - Lakukan iterasi berulang sampai tidak ada pertukaran yang dilakukan:
 - o Atur swapped ke false di awal setiap iterasi.
 - o Set current ke head dari list.
 - o Selama current.next tidak null (masih ada node berikutnya):
 - Bandingkan data pada node current dengan data pada node current.next.
 - Jika data pada current lebih besar dari data pada current.next, lakukan pertukaran:
 - Tukar data antara kedua node (bukan pointer).
 - Set swapped menjadi true untuk menunjukkan bahwa ada pertukaran yang dilakukan.
 - Pindahkan current ke node berikutnya (current = current.next).
3. Pengulangan:
 - Ulangi langkah 2 sampai tidak ada lagi pertukaran yang dilakukan (artinya list sudah terurut).

Contoh Proses Bubble Sort

- List awal : 4 – 2 – 3 – 1 dan akan melakukan sorting membesar / ascending
- Iterasi pertama:
 - o Bandingkan 4 dan 2: $4 > 2$, lakukan penukaran, 2 – 4 – 3 – 1
 - o Bandingkan 4 dan 3: $4 > 3$, lakukan penukaran, 2 – 3 – 4 – 1
 - o Bandingkan 4 dan 1: $4 > 1$, lakukan penukaran, 2 – 3 – 1 – 4
 - o Kondisi list di akhir iterasi: 2 – 3 – 1 – 4
- Iterasi kedua:
 - o Bandingkan 2 dan 3: $2 < 3$, tidak terjadi penukaran
 - o Bandingkan 3 dan 1: $3 > 1$, lakukan penukaran, 2 – 1 – 3 – 4
 - o Bandingkan 3 dan 4: $3 < 4$, tidak terjadi penukaran
 - o Kondisi list di akhir iterasi: 2 – 1 – 3 – 4

- Iterasi ketiga:

- o Bandingkan 2 dan 1: $2 > 1$, lakukan penukaran, 1 – 2 – 3 – 4
- o Bandingkan 2 dan 3: $2 < 3$, tidak terjadi penukaran
- o Bandingkan 3 dan 4 : $3 < 4$, tidak terjadi penukaran
- o Kondisi list di akhir iterasi: 1 – 2 – 3 – 4

Sub-Program:

Procedure bubbleSort(in/out L : list)
{ I.S. List tidak kosong.
F.S. elemen pada listurut membesar berdasarkan infonya}

Code:

```
#include <iostream>
using namespace std;

// Struktur node pada linked list
struct Node {
    int data;
    Node* next;
};

// Fungsi untuk menambahkan elemen ke dalam linked list
void append(Node** head_ref, int new_data) {
    Node* new_node = new Node(); // Membuat node baru
    Node* last = *head_ref; // Merujuk ke head

    new_node->data = new_data;
    new_node->next = nullptr; // Set node terakhir

    if (*head_ref == nullptr) {
        *head_ref = new_node; // Jika list kosong, node baru menjadi head
        return;
    }

    while (last->next != nullptr) { // Mencari node terakhir
        last = last->next;
    }

    last->next = new_node; // Menghubungkan node baru ke akhir list
}

// Fungsi untuk mencetak elemen dalam linked list
void printList(Node* node) {
    while (node != nullptr) {
        cout << node->data << " ";
        node = node->next;
    }
    cout << endl;
}
```

```

// Prosedur untuk mengurutkan linked list menggunakan bubble sort
void bubbleSort_2311104002(Node* head) {
    bool swapped;
    Node* current;
    Node* last_ptr = nullptr; // Digunakan untuk mempercepat proses

    if (head == nullptr) // Jika list kosong
        return;

    do {
        swapped = false;
        current = head;

        while (current->next != last_ptr) {
            if (current->data > current->next->data) {
                // Tukar data antar node
                int temp = current->data;
                current->data = current->next->data;
                current->next->data = temp;
                swapped = true;
            }
            current = current->next;
        }
        last_ptr = current; // Mengurangi iterasi dengan menempatkan last_ptr
    } while (swapped); // Ulangi sampai tidak ada pertukaran
}

int main() {
    Node* head = nullptr; // Membuat linked list kosong

    // Memasukkan 5 elemen ke dalam linked list
    int n;
    for (int i = 0; i < 5; i++) {
        cout << "Masukkan elemen integer: ";
        cin >> n;
        append(&head, n);
    }

    cout << "List sebelum sorting: ";
    printList(head);

    // Panggil prosedur bubbleSort untuk mengurutkan elemen
    bubbleSort_2311104002(head);

    cout << "List setelah sorting: ";
    printList(head);

    return 0;
}

```

- Terdapat **Struct Node** berfungsi membuat node dalam **sigle linked list** yaitu setiap node terdapat **data** dan **next** sebagai pointer ke node berikutnya.
- Fungsi **append** yaitu untuk menambahkan elemen baru ke akhir linked list
- Fungsi **printList** digunakan untuk mencetak elemen-elemen dalam linked list
- Prosedur **bubble sort** untuk mengurutkan elemen mulai dari yang terkecil ke terbesar

Output:

```

Masukkan elemen integer: 12
Masukkan elemen integer: 26
Masukkan elemen integer: 15
Masukkan elemen integer: 35
Masukkan elemen integer: 8
List sebelum sorting: 12 26 15 35 8
List setelah sorting: 8 12 15 26 35

Process returned 0 (0x0)   execution time : 41.237 s
Press any key to continue.

```

- Pengguna diminta memasukkan 5 elemen integer. Setelah itu akan menampilkan list sebelum dan setelah diurutkan menggunakan bubble sort

Soal 3: Menambahkan Elemen Secara Terurut

Deskripsi Soal: Buatlah program yang mengizinkan pengguna memasukkan 4 elemen integer ke dalam list secara manual. Kemudian, minta pengguna memasukkan elemen tambahan yang harus ditempatkan di posisi yang sesuai sehingga list tetap terurut.

Instruksi

1. Implementasikan procedure **insertSorted** untuk menambahkan elemen baru ke dalam list yang sudah terurut.
2. Tampilkan list setelah elemen baru dimasukkan.

Sub-Program:

```
Procedure insertSorted( in/out L : list, in P : address)
{ I.S. List tidak kosong.
  F.S. Menambahkan elemen secara terurut}
Dictionary
  Q, Prev: address
  found: bool
Algorithms
  Q ← L.head
  found ← false

  //melakukan perulangan selama found masih false dan Q masih menunjuk elemen pada list
  While .....
    //melakukan pengecekan apakah info dari elemen yang ditunjuk memiliki nilai lebih
    kecil dari pada P
    if ....
      //jika iya maka Prev diisi elemen Q, dan Q diisi elemen setelahnya
      ....
      //jika tidak maka isi found dengan nilai 'true'
      else
        . . .
      Endif
      //lakukan perpindahan Q
      ....
    endwhile

    //melakukan pengecekan apakah Q elemen head
    if ....
      //jika iya, maka tambahkan P sebagai head
      ....
      //melakukan pengecekan apakah Q berisi null (sudah tidak menunjuk elemen pada list
      else if ...
        //jika iya, maka tambahkan P sebagai elemen terakhir
        ...
        //jika tidak keduanya, maka tambahkan P pada posisi diantara Prev dan Q
        else
          ....
        endif
      endif
    endwhile
  endprocedure
```

Code:


```

#include <iostream>
using namespace std;

// Struktur node pada linked list
struct Node {
    int data;
    Node* next;
};

// Fungsi untuk menambahkan elemen ke dalam linked list secara terurut
void insertSorted_2311104002(Node** head_ref, int new_data) {
    Node* new_node = new Node(); // Membuat node baru
    new_node->data = new_data;
    new_node->next = nullptr;

    // Jika list kosong atau elemen baru lebih kecil dari elemen pertama
    if (*head_ref == nullptr || (*head_ref->data >= new_node->data) {
        new_node->next = *head_ref;
        *head_ref = new_node; // Elemen baru menjadi head
    } else {
        Node* current = *head_ref;

        // Cari posisi yang tepat untuk menvisipkan elemen baru
        while (current->next != nullptr && current->next->data < new_node->data) {
            current = current->next;
        }

        // Menvisipkan elemen baru di antara current dan current->next
        new_node->next = current->next;
        current->next = new_node;
    }
}

// Fungsi untuk mencetak elemen dalam linked list
void printList(Node* node) {
    while (node != nullptr) {
        cout << node->data << " ";
        node = node->next;
    }
    cout << endl;
}

int main() {
    Node* head = nullptr; // Membuat linked list kosong

    // Memasukkan 4 elemen secara terurut ke dalam linked list
    int n;
    for (int i = 0; i < 4; i++) {
        cout << "Masukkan elemen integer: ";
        cin >> n;
        insertSorted_2311104002(&head, n); // Menambahkan elemen secara terurut
    }

    cout << "List sebelum menambah elemen baru: ";
    printList(head);

    // Meminta pengguna memasukkan elemen tambahan
    int new_val;
    cout << "Masukkan elemen tambahan yang akan disisipkan secara terurut: ";
    cin >> new_val;

    // Menvisipkan elemen tambahan secara terurut
    insertSorted_2311104002(&head, new_val);

    cout << "List setelah menambah elemen baru: ";
    printList(head);

    return 0;
}

```

- Fungsi **insertSorted(Node** head_ref, int new_data)**: Menambahkan elemen baru dalam list, Jika lebih kecil dari elemen pertama maka menjadi head Jika tidak maka traversal dilakukan agar menemukan posisi yang tepat untuk menyiapkan elemen baru.
- Fungsi **printList(Node* node)** : Digunakan untuk mencetak elemen-elemen dalam linked list dari awal hingga akhir.

- Fungsi **insertSorted** digunakan untuk menambahkan elemen baru secara terurut ke dalam list yang sudah ada.

Output:

```
Masukkan elemen integer: 4
Masukkan elemen integer: 2
Masukkan elemen integer: 6
Masukkan elemen integer: 10
List sebelum menambah elemen baru: 2 4 6 10
Masukkan elemen tambahan yang akan disisipkan secara terurut: 8
List setelah menambah elemen baru: 2 4 6 8 10
```

- Pengguna memasukkan 4 elemen integral secara manual, Selanjutnya memasukkan elemen tambahan dan program akan menampilkan list sebelum dan sesudah elemen baru disisipkan.

Latihan (Modul)

1. Buatlah ADT Single Linked list sebagai berikut di dalam file “singlelist.h”:

```
Type infotype : int
Type address : pointer to ElmList
Type ElmList <
    info : infotype
    next : address
>
Type List : < First : address >
prosedur CreateList( in/out L : List )
fungsi alokasi( x : infotype ) : address
prosedur dealokasi( in/out P : address )
prosedur printInfo( in L : List )
prosedur insertFirst( in/out L : List, in P : address )
```

Kemudian buat implementasi ADT Single Linked list pada file “singlelist.cpp”.

Adapun isi data



Cobalah hasil implementasi ADT pada file “main.cpp”

```
int main()
{
    List L;
    address P1, P2, P3, P4, P5 = NULL;
    createList(L);

    P1 = alokasi(2);
    insertFirst(L,P1);

    P2 = alokasi(0);
    insertFirst(L,P2);

    P3 = alokasi(8);
    insertFirst(L,P3);

    P4 = alokasi(12);
    insertFirst(L,P4);

    P5 = alokasi(9);
    insertFirst(L,P5);

    printInfo(L)
    return 0;
}
```

Code :

```

#include <iostream>
using namespace std;

typedef int infotype;
typedef struct ElmList* address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

// Prosedur untuk membuat list kosong
void createList(List &L);

// Fungsi untuk mengalokasikan elemen baru
address alokasi(infotype x);

// Prosedur untuk dealokasi elemen
void dealokasi(address &P);

// Prosedur untuk mencetak elemen list
void printInfo(List L);

// Prosedur untuk menvisipkan elemen di awal list
void insertFirst(List &L, address P);

// Fungsi untuk mencari elemen berdasarkan info
address findElm(List L, infotype x);

// Fungsi untuk menghitung total info dari semua elemen
int totalInfo(List L);

#endif

```

singlelist.h

- Mendefinisikan tipe data **infotype**, **address** serta struktur **ElmList**, **List**. Digunakan untuk deklarasi dari fungsi dan prosedur membuat list, mengalokasikan serta dealokasi elemen , menambahkan elemen pada awal list, mencari elemen dan mencetak list

```

#include "singlelist.h"

// Prosedur untuk membuat list kosong
void createList(List &L) {
    L.First = nullptr;
}

// Fungsi untuk mengalokasikan elemen baru
address alokasi(infotype x) {
    address P = new ElmList;
    if (P != nullptr) {
        P->info = x;
        P->next = nullptr;
    }
    return P;
}

// Prosedur untuk dealokasi elemen
void dealokasi(address &P) {
    delete P;
    P = nullptr;
}

// Prosedur untuk mencetak elemen-elemen dalam list
void printInfo(List L) {
    address P = L.First;
    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }
    cout << endl;
}

// Prosedur untuk menvisipkan elemen di awal list
void insertFirst(List &L, address P) {
    if (P != nullptr) {
        P->next = L.First;
        L.First = P;
    }
}

// Fungsi untuk mencari elemen berdasarkan info
address findElm(List L, infotype x) {
    address P = L.First;
    while (P != nullptr) {
        if (P->info == x) {
            return P;
        }
        P = P->next;
    }
    return nullptr; // Jika tidak ditemukan
}

// Fungsi untuk menghitung total info dari semua elemen
int totalInfo(List L) {
    address P = L.First;
    int total = 0;
    while (P != nullptr) {
        total += P->info;
        P = P->next;
    }
    return total;
}

```

singlelist.cpp

- Fungsi **create** yaitu membuat list kosong, **alokasi** untuk mengalokasikan memori untuk node baru, **dealokasi** membebaskan memori yang dialokasikan untuk node, fungsi **insertFirst** untuk menambahkan elemen baru pada awal list, **findElm** untuk mencari elemen berdasarkan info dan **totalInfo** untuk menghitung total info dari semua elemen pada list.

```

#include "singlelist.h"

int main() {
    List L;
    address P1, P2, P3, P4, P5 = nullptr;

    // Membuat list kosong
    createList(L);

    // Menambahkan elemen-elemen ke dalam list
    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    // Menampilkan list
    cout << "Elemen dalam list: ";
    printInfo(L);

    // Mencari elemen dengan info 8
    address found = findElm(L, 8);
    if (found != nullptr) {
        cout << "Elemen dengan info 8 ditemukan." << endl;
    } else {
        cout << "Elemen dengan info 8 tidak ditemukan." << endl;
    }

    // Menghitung total info dari semua elemen
    int total = totalInfo(L);
    cout << "Total nilai elemen dalam list: " << total << endl;

    return 0;
}

```

main.cpp

- Berfungsi membuat list kosong serta menambahkan beberapa elemen dalam list menggunakan **insertFirst**, mencetak elemen dalam list, mencari elemen dengan info 8 menggunakan **findElm** dan menghitung serta menampilkan total info dari semua elemen pada list

Output:

```

Elemen dalam list: 9 12 8 0 2
Elemen dengan info 8 ditemukan.
Total nilai elemen dalam list: 31

Process returned 0 (0x0)   execution time : 0.114 s
Press any key to continue.

```

- Program ini implementasi dari Single Linked List menggunakan prosedur dasar untuk menambahkan elemen pada awal list serta mencari elemen dari info dan menghitung total nilai info dari semua elemen